



مشروع البرمجة التفرعية

تقدمة الطلاب :

نور الدين باسل خليفة

ياسر عمار الطاووس

خليل قاسم الدنيفات

عبد الرحمن نبيل دادو

معاذ عبد الرؤوف حمدان

❖ التقنيات المستخدمة :

Laravel , mysql , redis , nginx

❖ الطرق المستخدمة في تجارب الاختبار :

محاكاة ١٠٠ مستخدم jmeter

تخصيص benchmarks table في قاعدة البيانات بحيث يوجد

benchmarks middleware يقوم بحساب ال

request لكل cpu time , max ram usage , reponse time تصل للسيرفر .

❖ الطلب الأول :

تم حل مشكلة race condition عن طريق تطبيق قفل تشائمي

pessimistic lock عن طريق تعليمة db::transaction ، و هذا النوع من الاقفال يقوم بالسماح بتعديل واحد فقط على البيانات الموجودة داخله مما يمنع حدوث مشكلة اكثر من تعديل في وقت واحد ، في الصورة الاولى الحالة الابتدائية حيث يوجد من مخزون المنتج ١٠٠ و لا يوجد طلبات بعد ، في الصورة الثانية تم محاكاة طلبات ل ١٠٠ مستخدم بعدد ٣ قطع لكل مستخدم اي المجموع ٣٠٠ قطعة و تم تسجيل جميع الطلبات مع بقاء ٧ قطع في المخزون اي ان البيانات لم تعد حقيقية ، في الصورة الثالثة بعد وضع القفل تم تسجيل ٣٣ طلب بمجموع ٩٩ قطعة و بقي في المخزون قطعة واحدة و تم رفض باقي الطلبات

```
public function orderCart(Request $request)
{
    $user = $request->user();
    $carts = $user->carts()->with('product')->get();

    if ($carts->isEmpty()) {
        return response()->json(['message' => 'Cart is empty'], 400);
    }

    foreach ($carts as $cart) {
        if ($cart->product->inventory < $cart->quantity) {
            return response()->json([
                'message' => "Insufficient inventory for {$cart->product->name}",
            ], 400);
        }
    }

    $total = $carts->sum(function ($item) {
        return $item->quantity * $item->product->price;
    });

    if ($user->wallet < $total) {
        return response()->json(['message' => 'Insufficient wallet balance'], 400);
    }

    //DB::transaction create a pessimistic lock
    return DB::transaction(function () use ($user, $carts, $total) {
        $order = Order::create([
            'user_id' => $user->id,
            'total' => $total,
            'status' => 'completed',
        ]);

        foreach ($carts as $cart) {
            OrderItem::create([
                'order_id' => $order->id,
                'product_id' => $cart->product_id,
                'quantity' => $cart->quantity,
                'price' => $cart->product->price,
            ]);

            $cart->product->decrement('inventory', $cart->quantity);
        }

        $user->decrement('wallet', $total);
        $user->carts()->delete();

        OrderCompletedNotification::dispatchSync($user->id, (float) $total);
        CreateInvoice::dispatchSync($user->id, (float) $total);

        return response()->json(['message' => 'Order placed successfully', 'order'
    ]);
    });
}
```

الحالة الابتدائية :

A-Z name ▼	123 price ▼	123 inventory ▼	123 total_order_quantity ▼
Laptop	699.99	100	0

قبل وضع القفل :

A-Z name ▼	123 price ▼	123 inventory ▼	123 total_order_quantity ▼
Laptop	699.99	7	300

بعد وضع القفل :

A-Z name ▼	123 price ▼	123 inventory ▼	123 total_order_quantity ▼
Laptop	699.99	1	99

❖ الطلب الثاني :

المحافظة على الموارد و التحكم بعدد العمليات المتوازية

قمنا بإضافة rate limiter يسمح لكل مستخدم بالقيام ب ٦٠ request لكل دقيقة و

هذا الرقم كافي لك المستخدمين و بنفس الوقت يمنع حدوث هجمات ddos و قمنا

بإضافة redis

aimd (additive increase / multiplicative decrease) redis based
concurrency limiter

Laravel يعمل بآلية process per request اي انه غير قادر على معرفة عدد العمليات

المتوازية التي تحصل لذلك نحتاج قاعدة بيانات redis بحيث تحتوي على عداد يقوم

بالازدياد كلما وصلت request جديدة الى السيرفر او حدث background job و عند انتهائه فانه ينقص و هذه الطريقة فعالة بحساب عدد العمليات المتوازية المسموحة بحيث لدينا مجال بين 0 و 100 و هذه الارقام فعالة في الفحوصات التي قمنا بها ولكن لكل حالة ارقام متغيرة

عند كل request يتم قياس سرعة استجابة السيرفر فاذا كانت بطيئة يتم زيادة عدد العمليات المتوازية بشكل تزايدي +1 و اذا كانت سريعة زيادة عن اللزوم يتم تخفيض نسبة العمليات المتوازية المسموحة الى 70%.

```
class AdaptiveResourceManager
{
    private string $limitKey = 'capacity:current_limit';
    private string $counterKey = 'capacity:active_count';
    private string $metricsKey = 'capacity:metrics';

    private int $minLimit = 5;
    private int $maxLimit = 100;
    private int $defaultLimit = 20;

    public function executeWithCapacityControl(callable $operation): mixed
    {
        $limit = $this->getCurrentLimit();
        $current = Redis::incr($this->counterKey);

        if ($current > $limit) {
            Redis::decr($this->counterKey);
            $this->recordRejection();
            throw new CapacityExceededException($current, $limit);
        }

        $start = microtime(true);

        try {
            $result = $operation();
            $this->recordSuccess(microtime(true) - $start);
            return $result;
        } catch (\Throwable $e) {
            $this->recordFailure();
            throw $e;
        } finally {
            Redis::decr($this->counterKey);
        }
    }

    private function recordSuccess(float $responseTime): void
    {
        Redis::lpush($this->metricsKey, json_encode([
            'status' => 'success',
            'time' => $responseTime,
            'ts' => now()->timestamp,
        ]));
        Redis::ltrim($this->metricsKey, 0, 99); // keep last 100 records

        // Slow increase: +1 when things are going well
        if ($responseTime < 1.0) {
            $newLimit = min($this->maxLimit, $this->getCurrentLimit() + 1);
            Redis::set($this->limitKey, $newLimit);
        }
    }

    private function recordRejection(): void
    {
        // Fast decrease: cut by 25% when at capacity
        $newLimit = max($this->minLimit, (int)($this->getCurrentLimit() * 0.75));
        Redis::set($this->limitKey, $newLimit);
    }

    private function recordFailure(): void
    {
        // Moderate decrease: cut by 10% on errors
        $newLimit = max($this->minLimit, (int)($this->getCurrentLimit() * 0.90));
        Redis::set($this->limitKey, $newLimit);
    }

    public function getCurrentLimit(): int
    {
        return (int)(Redis::get($this->limitKey) ?? $this->defaultLimit);
    }
}
```

❖ الطلب الثالث :

في laravel ال background jobs تكون خارج المسار الرئيسي لل requests

قمنا باضافة ٣ انواع من ال background jobs :

1. ارسال اشعار بتمام عملية الشراء
2. ارسال فاتورة
3. توليد تقرير يومي يحوي مجموع المبيعات و يتولد تلقائيا عند انتهاء اليوم

❖ الطلب الرابع :

قمنا بمعالجة ١٠٠٠٠٠ طلب مرة دفعة واحدة و مرة على ٢٠ دفعة ٥٠٠٠ لكل دفعة و كانت النتائج كالتالي: التقرير المولد على دفعات (السجل الثاني) اسرع بحوالي ١١ مرة من التقرير المولد دفعة واحدة (السجل الأول) و استهلك الرام الاعظمي نصف ما استهلكه التقرير المولد دفعة واحدة

```

public const BATCH_SIZE = 5000;
public ?string $date;
public function __construct(?string $date = null)
{
    $this->date = $date ?? now()->toDateString();
}
public function handle(): void
{
    $date = $this->date;

    $totalOrders = DB::table('orders')
        ->whereDate('created_at', $date)
        ->where('status', 'completed')
        ->count();

    $totalSales = 0;
    $offset = 0;

    while ($offset < $totalOrders) {
        $batch = DB::table('orders')
            ->whereDate('created_at', $date)
            ->where('status', 'completed')
            ->select('total')
            ->offset($offset)
            ->limit(self::BATCH_SIZE)
            ->get();

        foreach ($batch as $order) {
            $totalSales += $order->total;
        }

        $offset += self::BATCH_SIZE;
    }

    Report::updateOrCreate(
        ['date' => $date, 'type' => 'daily_sales_batch'],
        [
            'total_sales' => $totalSales,
            'total_orders' => $totalOrders,
        ]
    );
}

```

AZ name	123 cpu_time	123 peak_ram_usage
App\Jobs\GenerateDailyReport	700.8061	8
App\Jobs\GenerateDailyReport12	61.7909	4

❖ الطلب الخامس :

تم استعمال nginx لتوزيع الاحمال

تم vip pool و هذه مخصصة للزبائن المهمة ، عندما يقوم المستخدم بتسجيل الدخول يحصل على auth token و يحصل على is_vip cookie فاذا كان الزبون مهم يتم تحويله الى ال vip pool و اذا لم يكن مهم يتم تحويله الى ال regular pool .

يوجد سيرفرين في ال vip pool و الخوارزمية هي least connection و السبب هو ان ليس كل ال requests بنفس الحجم و الاستهلاك و بما ان هذه الخوارزمية تقيس

الـ active connections فتعتبر اكثر دقة بتحقيق توازن بين السيرفرات و سبب وجود سيرفرين هو اجراء احتياطي في حال وقوع احد السيرفرات.

يوجد ٤ سيرفرات في الـ regular pool و تم استعمال خوارزمية least connection مع وضع اوزان بحيث السيرفر الاول يتلقى نصف الـ requests التي يتم تلقيها من قبل باقي السيرفرات و السبب هو ان السيرفر الاول سوف يتم توليد التقرير اليومي عليه ، و هذه العملية تستهلك موارد و خوارزمية least connection لا تستطيع قياس هذا الاستهلاك لأن الـ background jobs منعزلة و لا تستطيع الخوارزمية ان تتواصل معها

```
map $http_cookie $is_vip {
    default 0;
    ~*isVip=1 1;
}

upstream regular_backend {
    least_conn;
    server 127.0.0.1:8002 weight=1;
    server 127.0.0.1:8003 weight=2;
    server 127.0.0.1:8004 weight=2;
    server 127.0.0.1:8005 weight=2;
    keepalive 64;
}

upstream vip_backend {
    least_conn;
    server 127.0.0.1:8000;
    server 127.0.0.1:8001;
    keepalive 64;
}

server {
    listen 9000;
    server_name _;

    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    location /api/ {
        set $backend regular_backend;
        if ($is_vip = 1) {
            set $backend vip_backend;
        }
        proxy_pass http://$backend;
    }

    location / {
        proxy_pass http://regular_backend;
    }
}
```

سرعة الاستجابة عند استخدام nginx :

A-Z name	123 avg_response_time
api/orders/cart-loadbalancer	31.376693

سرعة الاستجابة بدون استخدام nginx :

A-Z name	123 avg_response_time
api/orders/cart	38.25463023

<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8002]. Press Ctrl+C to stop the server 2026-05-17 22:58:13 /api/login ~ 1s 2026-05-17 22:58:15 /api/login ~ 0.10ms 2026-05-17 22:58:16 /api/login ~ 0.14ms 2026-05-17 22:58:17 /api/login ~ 500.37ms 2026-05-17 22:58:18 /api/login ~ 500.41ms 2026-05-17 22:58:18 /api/orders/cart ~ 500.40ms 2026-05-17 22:58:19 /api/login ~ 500.40ms 2026-05-17 22:58:20 /api/orders/cart ~ 0.16ms 2026-05-17 22:58:20 /api/orders/cart ~ 0.15ms 2026-05-17 22:58:20 /api/login ~ 0.16ms </pre>	<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8000]. Press Ctrl+C to stop the server 2026-05-17 22:58:14 /api/orders/cart ~ 500.38ms 2026-05-17 22:58:14 /api/orders/cart ~ 0.10ms 2026-05-17 22:58:14 /api/orders/cart ~ 500.47ms 2026-05-17 22:58:14 /api/orders/cart ~ 0.09ms 2026-05-17 22:58:15 /api/orders/cart ~ 0.06ms 2026-05-17 22:58:15 /api/orders/cart ~ 0.18ms 2026-05-17 22:58:16 /api/orders/cart ~ 0.18ms 2026-05-17 22:58:16 /api/orders/cart ~ 0.07ms 2026-05-17 22:58:16 /api/orders/cart ~ 0.14ms 2026-05-17 22:58:17 /api/orders/cart ~ 0.16ms </pre>
<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8003]. Press Ctrl+C to stop the server 2026-05-17 22:58:13 /api/login ~ 1s 2026-05-17 22:58:14 /api/login ~ 0.14ms 2026-05-17 22:58:14 /api/login ~ 0.17ms 2026-05-17 22:58:15 /api/login ~ 0.16ms 2026-05-17 22:58:15 /api/login ~ 0.14ms 2026-05-17 22:58:16 /api/login ~ 0.18ms 2026-05-17 22:58:17 /api/orders/cart ~ 0.20ms 2026-05-17 22:58:17 /api/orders/cart ~ 0.14ms 2026-05-17 22:58:18 /api/orders/cart ~ 0.07ms 2026-05-17 22:58:18 /api/login ~ 0.15ms 2026-05-17 22:58:18 /api/login ~ 501.13ms </pre>	<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8001]. Press Ctrl+C to stop the server 2026-05-17 22:58:14 /api/orders/cart ~ 500.44ms 2026-05-17 22:58:14 /api/orders/cart ~ 0.09ms 2026-05-17 22:58:14 /api/orders/cart ~ 1.62ms 2026-05-17 22:58:14 /api/orders/cart ~ 0.06ms 2026-05-17 22:58:15 /api/orders/cart ~ 500.61ms 2026-05-17 22:58:15 /api/orders/cart ~ 0.15ms 2026-05-17 22:58:15 /api/orders/cart ~ 0.17ms 2026-05-17 22:58:16 /api/orders/cart ~ 0.18ms 2026-05-17 22:58:16 /api/orders/cart ~ 0.13ms 2026-05-17 22:58:17 /api/orders/cart ~ 0.17ms </pre>
<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8004]. Press Ctrl+C to stop the server 2026-05-17 22:58:13 /api/login ~ 500.43ms 2026-05-17 22:58:14 /api/login ~ 500.38ms 2026-05-17 22:58:14 /api/login ~ 500.54ms 2026-05-17 22:58:14 /api/login ~ 0.15ms 2026-05-17 22:58:15 /api/login ~ 500.48ms 2026-05-17 22:58:16 /api/login ~ 500.40ms 2026-05-17 22:58:17 /api/login ~ 500.39ms 2026-05-17 22:58:17 /api/login ~ 500.42ms 2026-05-17 22:58:17 /api/login ~ 0.09ms 2026-05-17 22:58:18 /api/orders/cart ~ 0.10ms 2026-05-17 22:58:18 /api/orders/cart ~ 0.09ms </pre>	<pre> laravel-e-commerce-pp-sanctum : php INFO Server running on [http://127.0.0.1:8005]. Press Ctrl+C to stop the server 2026-05-17 22:58:13 /api/login ~ 1s 2026-05-17 22:58:14 /api/login ~ 0.07ms 2026-05-17 22:58:15 /api/login ~ 500.35ms 2026-05-17 22:58:16 /api/login ~ 500.44ms 2026-05-17 22:58:16 /api/login ~ 500.43ms 2026-05-17 22:58:17 /api/orders/cart ~ 0.13ms 2026-05-17 22:58:17 /api/login ~ 500.43ms 2026-05-17 22:58:18 /api/orders/cart ~ 0.14ms 2026-05-17 22:58:18 /api/login ~ 0.15ms 2026-05-17 22:58:19 /api/orders/cart ~ 0.11ms 2026-05-17 22:58:19 /api/orders/cart ~ 0.06ms </pre>

الطلب 6

تم استعمال ريديس كاش لتخزين المنتجات عند طلب صفحة المنتج و يتم الاحتفاظ بالمنتج لمدة عشر دقائق في الكاش و تتجدد كلما تم طلب صفحة المنتج مرة اخرى

الطلب 7

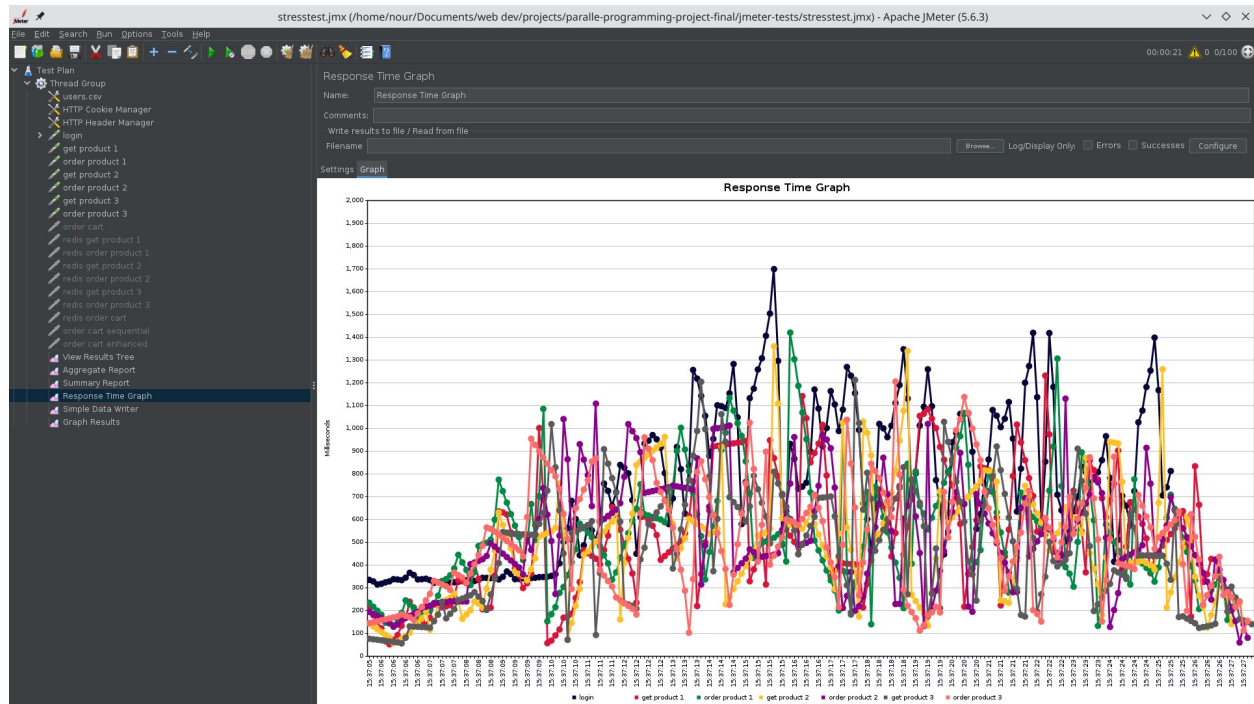
تم وضع قفل تشائي عند شراء منتج و قفل تفائلي عند شراء العربة و تم وضع نفس اختبارات الطلب الاول لضمان سلامة البيانات

الطلب 8

تم وضع جميع العمليات الاساسية داخل تعليمة [db::trascation](#) بحيث تنجح كلها او تفشل كلها و جميع العمليات الثانوية مثلا عمليات الخلفية داخل try catch بحيث فقط اذا نجحت العمليات الاساسية جميعها يتم تفعيل العمليات الثانوية و الا يفشل كل شيء معا بدون اي تغيير

الطلب 9

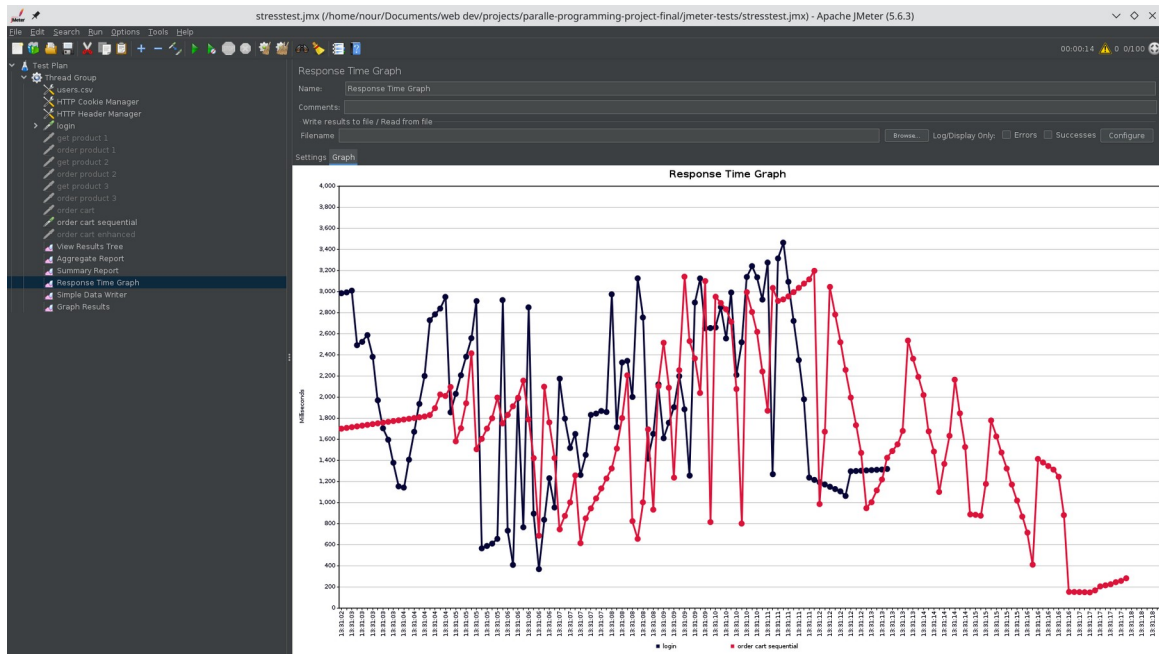
تم اختبار ضغط يحوي على جميع العمليات و كانت النتائج كالتالي



الطلب 10

تم ايجاد اختناق sequential database calls
و من ثم حله عن طريق concurrent database calls

قبل



بعد

